

Justificación De Estructuras De Datos

Decisiones de diseño y selección de estructuras

Sistema universitario desarrollado en Java 17 para gestionar estudiantes, materias, horarios, rutas de campus, reportes académicos y operaciones de deshacer/rehacer.

1. Enfoque Arquitectónico

El proyecto adopta una arquitectura simplificada con separación por paquetes: modelo para entidades, estructuras para la lógica central y excepciones para errores personalizados. Esta decisión facilita explicar responsabilidades y evita mezclar menú, reglas de negocio y datos.

2. Justificación De Estructuras

Estructura	Justificación
HashMap<String, Estudiante>	Permite buscar estudiantes por ID en tiempo promedio $O(1)$, ideal para consultas frecuentes.
HashMap<String, Materia>	Permite localizar materias por código de forma rápida y directa.
TreeMap<String, Aula>	Mantiene las aulas ordenadas por nombre, útil para listarlas de forma clara.
LinkedList<Materia>	Se usa para pre-requisitos e historial académico porque permite agregar elementos de forma sencilla.
Queue<Estudiante>	Representa naturalmente la cola de espera: primero en entrar, primero en salir.
Stack<Accion>	Modela deshacer/rehacer porque la última acción realizada es la primera que debe revertirse.
Stack<String>	Guarda reportes ya generados para navegación simple hacia atrás.
boolean[7][24]	Representa disponibilidad semanal de aulas con acceso directo por día y hora.
Double[10][20]	Representa notas por semestre y posición de materia con límites claros.
int[5][5]	Representa el grafo fijo del campus como matriz de adyacencia.

3. Por Qué Usar Java Collections

Se eligieron estructuras nativas de Java para reducir complejidad accidental. En lugar de implementar manualmente listas, pilas y colas, el proyecto se concentra en aplicar correctamente las estructuras al problema: inscripción, espera, reportes, rutas y deshacer/rehacer.

- El código queda más corto y fácil de mantener.
- Las estructuras son confiables porque pertenecen a la API estándar de Java.
- La sustentación puede enfocarse en por qué se usa cada estructura y no en errores internos de implementación.

4. Matrices Nativas

Las matrices se mantienen como arreglos nativos porque el problema tiene tamaños fijos: 7 días, 24 horas, 10 semestres, 20 materias por semestre y 5 edificios. Esto hace que el acceso sea directo y muy eficiente.

Matriz	Uso
boolean[7][24]	Control de disponibilidad de aulas.

Double[10][20]	Registro de notas por semestre.
Materia[10][20]	Relación entre cada nota y su materia.
int[5][5]	Distancias entre edificios para Dijkstra.

5. Dijkstra Con Grafo Fijo

El algoritmo de Dijkstra se aplica sobre una matriz de adyacencia `int[5][5]`. Cada posición `matrizDistancias[i][j]` indica la distancia entre dos edificios. Si no hay conexión directa, se usa un valor grande llamado INF.

- El arreglo `distancias` guarda la mejor distancia conocida desde el origen.
- El arreglo `anteriores` permite reconstruir la ruta final.
- El arreglo `visitados` evita procesar dos veces el mismo edificio.
- En cada paso se selecciona el edificio no visitado con menor distancia acumulada.

6. Deshacer/Rehacer Con Accion

En lugar de clonar todo el estado del sistema, se guarda una clase ligera `Accion`. Esta clase conserva el tipo de operación y los datos mínimos necesarios para aplicar la operación inversa.

Esta decisión reduce memoria, simplifica el código y es suficiente para operaciones como inscripción, cancelación, eliminación, registro de notas y cambios de horario.

7. Manejo De Excepciones

Las excepciones personalizadas hacen explícitas las reglas de negocio. En el menú principal se capturan con `try-catch` para evitar que el programa se cierre ante errores esperados.

Excepción	Cuándo se usa
<code>EstudianteNoEncontradoException</code>	Cuando no existe el ID consultado.
<code>PreRequisitoNoAprobadoException</code>	Cuando un estudiante no cumple los pre-requisitos.
<code>CupoLlenoException</code>	Cuando una materia no tiene cupos disponibles.
<code>HorarioConflictivoException</code>	Cuando un aula ya está reservada.
<code>PilaDeshacerVacíaException</code>	Cuando no hay acciones para deshacer o rehacer.

8. Conclusión

La solución prioriza claridad, mantenimiento y explicación académica. Cada estructura se eligió porque se ajusta naturalmente a una regla del negocio: `HashMap` para búsqueda rápida, `Queue` para espera, `Stack` para deshacer/rehacer, `LinkedList` para listas académicas y matrices para datos de tamaño fijo.